AIE 1902: Quantitative Methods with AI tools for Financial Markets

Slide 5: Machine Learning for Return Prediction

Ka Wai Tsang

The Chinese University of Hong Kong, Shenzhen

Outline

Can We Predict Returns?

The Ultimate Question

Is stock return prediction possible with machine learning?

The Challenge:

- Markets are efficient (mostly): Any information we can use is already in the current price.
- Signal-to-noise ratio is low: Even when there is some predictability, it tends to be small compared with the random fluctuations.
- Data look noisy: Returns are mostly random, so the model often picks up spurious patterns that do not repeat.
- Many possible features: Hundreds of technical indicators, fundamentals, or lags make it easy to find patterns in-sample that disappear out-of-sample.
- Complex models: ML models (trees, neural nets) can memorize training data, including pure noises, making overfitting likely.
- Temptation to tweak: Researchers and practitioners may keep changing features until in-sample performance looks good.

Today's Approach

What We'll Cover:

1. Feature engineering for finance
2. ML models

Key Principle:

- Out-of-sample performance is what matters
- Be honest about limitations
- Most attempts fail - that's OK

What is Feature Engineering?

Definition

Creating predictor variables (features) from raw data

Why It Matters:

- More important than model choice!
- 80% of ML work is feature engineering
- Good features $\gg$ sophisticated models

For Financial Data:

- Price-based features
- Technical indicators
- Fundamental ratios
- Macro indicators
- Interaction terms

The Information Paradox

If public ML worked, everyone would be rich

Methods (XGBoost, etc.) are widely available. The edge comes from **information**, not algorithms.

Reality:

- Useful information is **hard to attain** and **valuable**
- Public data (prices, volume) $\rightarrow$ arbitraged away quickly
- Features from easily accessible data $\rightarrow$ limited predictive power
- Edge requires: **novel data**, **speed**, or **superior processing**

Implication: Focus on *how* to attain and construct features, not just which model to use.

Sources of Features: The Data Hierarchy

Tier 1 — Public & Free:

- Prices, volume, splits (Yahoo, akshare, etc.)
- No edge by itself; everyone has it

Tier 2 — Public but Requires Effort:

- Fundamentals (financials, ratios)
- Macro (rates, VIX, economic releases)
- Requires cleaning, alignment, lag handling

Tier 3 — Proprietary / Alternative:

- News sentiment, social media, web traffic
- Order flow, dark pool data
- Satellite, credit card, geolocation
- Expensive or exclusive; where edge often lies

How to Attain Features

Public Data:

- APIs: Yahoo Finance, Alpha Vantage, akshare (free/limited)
- Financial databases: Wind, Bloomberg, Refinitiv (paid)

Practical Steps:

- Start with public data; learn the pipeline
- Invest in **processing** (cleaning, alignment) before buying expensive data
- Document data lineage, update frequency, survivorship
  - Data lineage: When results change, you should know which data or transformation caused it. Also for audits and reproducibility.
  - Update frequency: Your features must match your trading horizon. A monthly feature in a daily strategy can create lags.
  - Survivorship bias: Backtests on "current" stocks miss delisted, bankrupt, or merged names. Returns and Sharpe ratios are overstated.

How to Construct Useful Features

1. **Domain Knowledge**
   - Use finance theory (e.g., factor models, term structure)
   - Economic rationale $\rightarrow$ more robust than data mining
2. **Transformations**
   - Ratios, log returns, standardized scores
   - Rolling windows, lags, differences
   - Interaction terms (e.g., volatility $\times$ momentum)
3. **Novelty & Speed**
   - Lead time: feature available before market reacts?
   - Freshness: how quickly can you compute and trade?
   - Unique combination: new angle on existing data

PCA-like Methods: Setup

Although there are challenges with machine learning for financial data (low signal-to-noise ratio, overfitting, etc.), we can ease the issues with some statistical methods.

Data: p features $X_1, \ldots, X_p$ (e.g., stock characteristics). Stack as $\mathbf{X} \in \mathbb{R}^{n \times p}$.

Latent factor model: Each feature driven by m common factors $F_1, \ldots, F_m$ plus idiosyncratic noise (uncorrelated with other noises and factors):

$$X_j = \lambda_{j1} F_1 + \cdots + \lambda_{jm} F_m + E_j = \boldsymbol{\lambda}_j^\top \mathbf{F} + E_j$$

In matrix form: $\mathbf{X} = \mathbf{F}\boldsymbol{\Lambda}^\top + \mathbf{E}$, where $\boldsymbol{\Lambda} \in \mathbb{R}^{p \times m}$ (loadings), $\mathbf{F} \in \mathbb{R}^{n \times m}$ (factors).

Goal: Recover $\mathbf{F}$ (and $\boldsymbol{\Lambda}$) from $\mathbf{X}$. Different methods impose different assumptions and objectives.

PCA: Objective and Solution

PCA assumes factors are linear combinations of $\mathbf{X}$: $\mathbf{F} = \mathbf{XW}$, $\mathbf{W} \in \mathbb{R}^{p \times m}$.

Objective 1 (maximize variance): First PC

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{Xw})$$

Subsequent PCs: same, but orthogonal to previous ones.

Objective 2 (minimize reconstruction error):

$$\min_{\mathbf{W}:\mathbf{W}^\top \mathbf{W}=\mathbf{I}} \|\mathbf{X} - \mathbf{XWW}^\top\|_F^2$$

Solution: $\mathbf{W}$ = eigenvectors of $\widehat{\mathbf{\Sigma}} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$ corresponding to top m eigenvalues.

Equivalence: The two objectives yield the **same W** (same PCs).

Factor Analysis

Model: $\mathbf{X} = \mathbf{F}\boldsymbol{\Lambda}^\top + \mathbf{E}$, $\mathbf{F} \sim N(0, \mathbf{I})$, $\mathbf{E} \sim N(0, \boldsymbol{\Psi})$ (diagonal).

Recovery: Maximize likelihood $\rightarrow$ estimate $\boldsymbol{\Lambda}$, $\boldsymbol{\Psi}$; rotation for interpretability.

PCA vs FA:

	PCA	**FA**
Objective	Maximize variance	Latent factor model
Assumptions	None	Gaussian $\mathbf{F}$, $\mathbf{E}$
Use	Dimensionality reduction	Interpretable latent factors

Independent Component Analysis: Model and Goal

Model: $\mathbf{X} = \mathbf{SA}$, where $\mathbf{S} = (S_1, \ldots, S_m)^\top$ are **independent** (unknown) sources.

Goal: Find unmixing $\mathbf{W}$ s.t. $\widehat{\mathbf{S}} = \mathbf{XW}$ recovers sources (up to sign, scale, permutation).

Key assumption: At most one source is Gaussian (otherwise identifiability fails).

Objective: $\max_{\mathbf{w}} \mathbb{E}[G(\mathbf{Xw})]$ s.t. $\mathrm{Var}(\mathbf{Xw}) = 1$.

- Contrast G (e.g. $G(y) = \log\cosh(y)$) is chosen such that the objective function is maximum when $\mathbf{Xw}$ is highly non-Gaussian.
- As linear combinations are likely to be more Guassian, maximally non-Gaussian $\Rightarrow$ recovers a source.

ICA: Why Maximize Non-Gaussianity? (CLT)

Central Limit Theorem: A sum of independent r.v.s tends to be **more Gaussian** than the individual terms.

In our setup: Each $X_j = a_{j1}S_1 + \cdots + a_{jm}S_m$ is a **mixture** of sources $\Rightarrow$ mixtures tend to be **more Gaussian**.

Reversal: The **least Gaussian** linear combinations of $\mathbf{X}$ are those that equal (a multiple of) a single source. So: maximize non-Gaussianity $\Rightarrow$ recover sources.

Summary: Mixtures = more Gaussian; sources = less Gaussian. ICA finds directions of **minimum** Gaussianity.

ICA vs PCA/FA

PCA/FA: Find **uncorrelated** factors. For Gaussian, uncorrelated $\Leftrightarrow$ independent.

ICA: Find **independent** factors. Stronger than uncorrelated; needs non-Gaussianity.

When ICA helps: Sources are non-Gaussian (speech, music, fat-tailed returns). Separation / blind source separation.

When PCA suffices: Factors near-Gaussian; or only variance/correlation matters.

Implementing ICA Recovery Example

Your AI Prompt

"Write Python code to demonstrate that ICA recovers independent non-Gaussian sources from mixtures. (1) Generate 2 independent sources: $S_1 \sim$ Laplace, $S_2 \sim$ Uniform. (2) Mix them: $\mathbf{X} = \mathbf{SA}$ with a 2×2 matrix $\mathbf{A}$. (3) Apply ICA and PCA. (4) Create a 2×2 plot: true sources, observed mixtures, ICA recovered vs true (should align on diagonal), PCA components vs true (does not recover)."

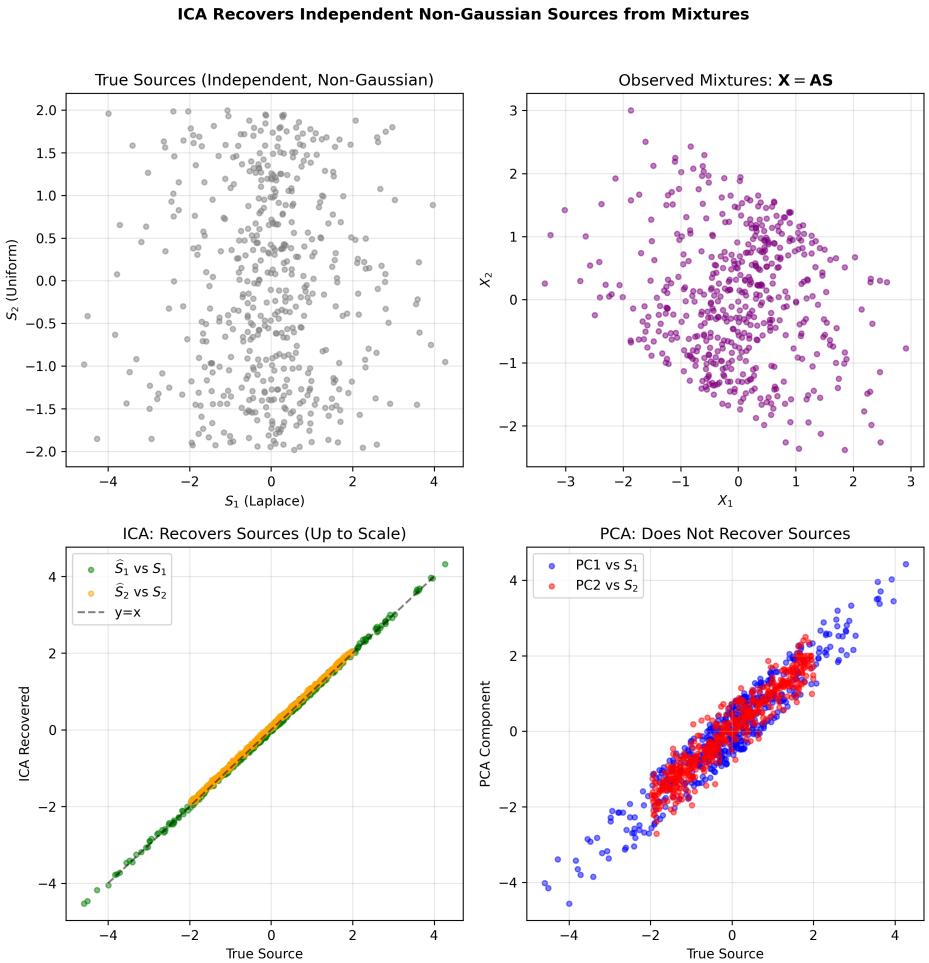
Key Lines to Check

```
S1 = np.random.laplace(0, 1, 500)
S2 = np.random.uniform(-2, 2, 500)
S = np.column_stack([S1, S2])
A = np.array([[1.5, 0.8], [-0.6, 1.2]])
X = (A @ S.T).T

X_scaled = StandardScaler().fit_transform(X)
ica = FastICA(n_components=2, random_state=42).fit(X_scaled)
pca = PCA(n_components=2).fit(X_scaled)
Z_ica = ica.transform(X_scaled)
Z_pca = pca.transform(X_scaled)
```

Illustrative Example: ICA Recovers Sources

Setup: $\mathbf{S} = (S_1, S_2)^\top$ independent ($S_1 \sim$ Laplace, $S_2 \sim$ Uniform). Mix: $\mathbf{X} = \mathbf{AS}$.



Top-left: True sources. **Top-right:** Observed mixtures (more Gaussian). **Bottom-left:** ICA recovers sources (points on diagonal). **Bottom-right:** PCA

Implementing PCA vs FA vs ICA Comparison

Your AI Prompt

"Write Python code to compare PCA, Factor Analysis, and ICA on stock return data. Download 8 stock returns (e.g., AAPL, MSFT, GOOGL, ...), standardize, then fit PCA(2), FactorAnalysis(2), and FastICA(2). Create a 2-panel plot: (1) scatter of Component 1 vs 2 for all three methods; (2) loadings (PC1, PC2, F1, F2, IC1, IC2) on original stocks. Save as plot_pca_fa_ica_comparison.png."

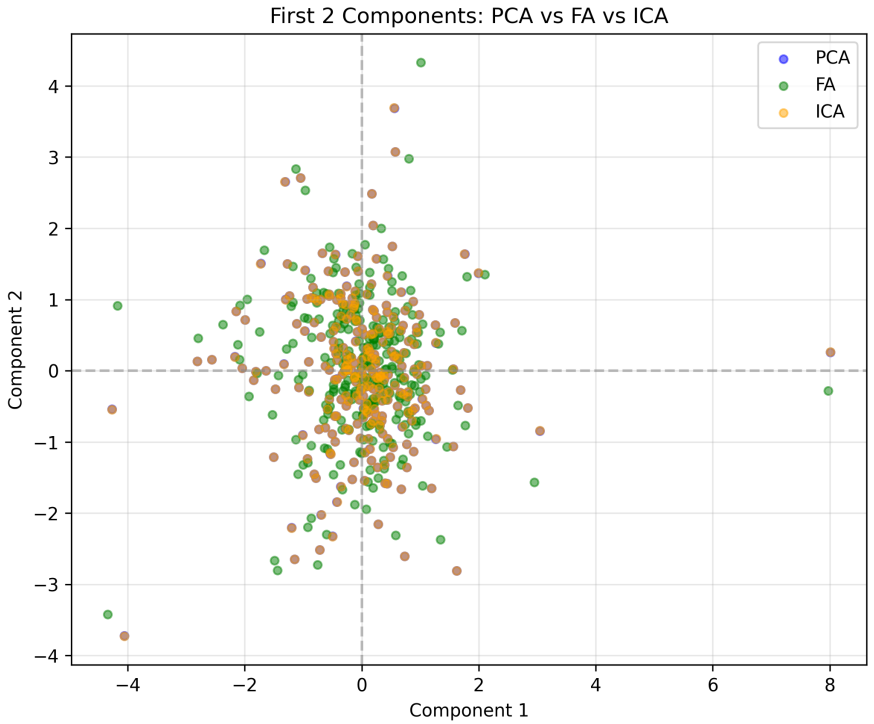
Key Lines to Check

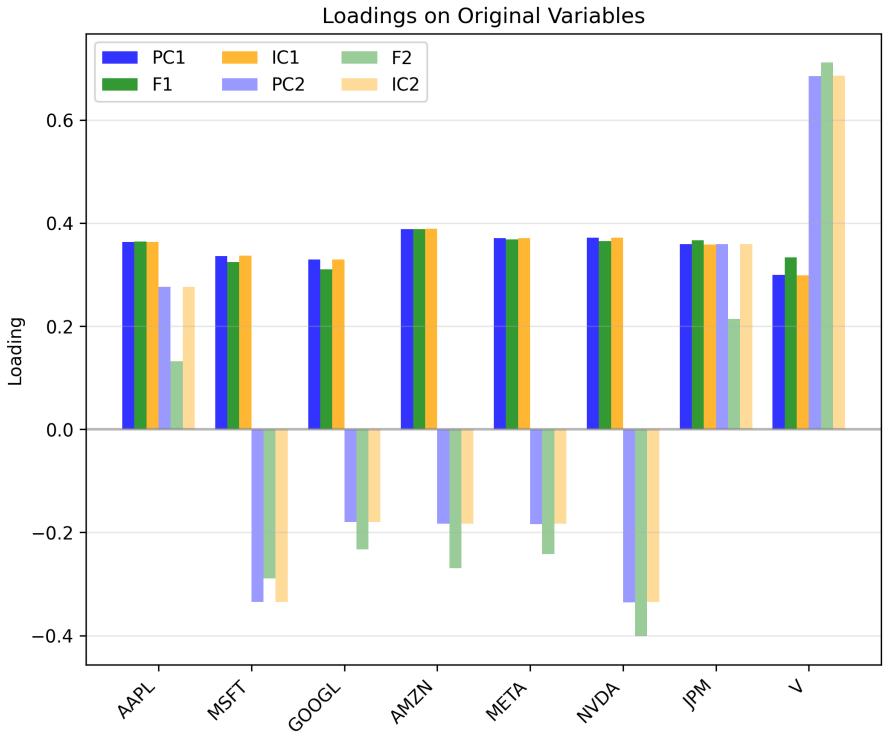
```
from sklearn.decomposition import PCA, FactorAnalysis, FastICA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(returns.values)
pca = PCA(n_components=2).fit(X_scaled)
fa = FactorAnalysis(n_components=2, random_state=42).fit(X_scaled)
ica = FastICA(n_components=2, random_state=42).fit(X_scaled)

Z_pca = pca.transform(X_scaled)
Z_fa = fa.transform(X_scaled)
Z_ica = ica.transform(X_scaled)
```

PCA vs FA vs ICA: Numerical Comparison





Left: Scatter of Component 1 vs 2. **Right:** Loadings on original variables. Run the script to reproduce; PCA, FA, ICA yield different factors.

PCA Variants & Extensions

Sparse PCA: Add sparsity on loadings:

$$\max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{X}\mathbf{w}) - \lambda\|\mathbf{w}\|_1$$

Fewer non-zero loadings $\rightarrow$ more interpretable.

Robust PCA: $\mathbf{X} = \mathbf{L} + \mathbf{S}$, $\mathbf{L}$ low-rank, $\mathbf{S}$ sparse (outliers):

$$\min_{\mathbf{L},\mathbf{S}} \|\mathbf{L}\|_* + \lambda\|\mathbf{S}\|_1 \quad \text{s.t. } \mathbf{X} = \mathbf{L} + \mathbf{S}$$

$\|\cdot\|_*$ = nuclear norm.

Kernel PCA: Map $\mathbf{X} \rightarrow \phi(\mathbf{X})$, apply PCA in feature space.
Dynamic factor models: $\mathbf{\Lambda}_t$ and/or $\mathbf{F}_t$ vary over time; Kalman filter for estimation.

Technical Indicators

Common Features:

Moving Averages:

- MA(5), MA(20), MA(50), MA(200)
- Crossover signals
- Distance from MA

Momentum:

- Returns over various periods (1d, 5d, 20d, 60d)
- RSI (Relative Strength Index)
- MACD

Volatility:

- Rolling std dev
- Realized volatility for intraday returns
- Volatility ratios

More Features

Volume Indicators:

- Volume MA
- Volume vs price
- On-Balance Volume (OBV, see next slide)

Fundamental Features:

- P/E ratio (price-to-earnings ratio)
- Book-to-market ("Book value of equity" to "Market capitalization")
- Debt ratios (e.g. Debt-to-equity, Debt-to-assets)
- Profitability metrics

Macro Features:

- Interest rates
- VIX (fear index)

On-Balance Volume (OBV)

Rule:

- Close $>$ previous close $\Rightarrow$ add today's volume to OBV
- Close $<$ previous close $\Rightarrow$ subtract today's volume from OBV
- Close $=$ previous close $\Rightarrow$ OBV unchanged

Formula: $\text{OBV}_t = \text{OBV}_{t-1} + \begin{cases} +V_t & \text{if } P_t > P_{t-1} \\ -V_t & \text{if } P_t < P_{t-1} \\ 0 & \text{if } P_t = P_{t-1} \end{cases}$ (V_t = volume, P_t = close)

Idea: Cumulative "signed" volume: up-day volume = buying pressure (add), down-day volume = selling pressure (subtract).

Use: Plot OBV with price. Rising OBV with price = trend confirmed; **divergence** (price new high, OBV not) $\Rightarrow$ weakening momentum, possible reversal.

Implementing Feature Engineering

Your AI Prompt

"Write Python code to create features from stock data including moving averages, momentum, volatility, volume, fundamentals, and macro (using akshare where possible)."

Data source: `akshare`. VIX: `yfinance` (`^VIX`) or similar.

Features: Where to Get or Compute Them

Price data → `ak.stock_us_daily(symbol, adjust="qfq")` [date, open, high, low, close, volume]

Intraday realized vol: `ak.stock_us_hist_min_em(symbol)` for minute data → sum of squared intraday returns.

Fundamentals (US): `ak.stock_financial_us_report_em(stock, symbol=..., indicator=...)` (balance sheet / income / cash flow; see akshare docs for `symbol` values) → debt, equity, earnings; `ak.stock_us_spot_em()` or `ak.stock_us_valuation_baidu()` for P/E, market cap → P/E, book-to-market, debt ratios, profitability.

Macro: `ak.macro_bank_usa_interest_rate()` → US interest rates. **VIX:** `yfinance` (`^VIX`) or other (not in akshare).

Data handling: Missing values (forward fill/drop), normalize if needed, create lag features, avoid look-ahead bias.

Avoiding Look-Ahead Bias

Common Mistakes:

WRONG:

- Use today's closing price to predict today's return
- Include future information in features
- Train on future-test data

RIGHT:

- Use only past data to predict future
- Lag features appropriately

Golden Rule

At time t, you can only use information available at time $t-1$!

Walk-Forward Validation

Expanding Window:

- Start with 2010-2015 to train
- Test on 2016
- Add 2016 to training data
- Test on 2017
- Repeat

Why This Matters:

- Models retrained periodically
- Parameters adapt over time
- More realistic than single split

Implementing Walk-Forward

Your AI Prompt

"Write Python code to implement walk-forward validation for time series with expanding window"

Key Steps:

1. Define start/end dates
2. Loop: select training window
3. Fit model on training data
4. Predict on next period
5. Evaluate predictions
6. Add test period to training
7. Repeat

Predicting Returns: Random Forest, XGBoost

Task: Predict return (continuous). Regression with tabular features (e.g. technicals, fundamentals).

Random Forest: Ensemble of decision trees; each tree sees a bootstrap sample and random subset of features. Handles non-linearity and interactions; built-in feature importance; relatively robust to overfitting.

XGBoost: Gradient boosting—sequentially add trees that fit residuals. Often best predictive performance; requires careful tuning (depth, learning rate) to avoid overfitting.

Weaknesses (both): Less interpretable than linear models; more hyperparameters; trees can overfit if too deep or too many.

When to use: Default choice for return prediction with many features when you care about accuracy. Prefer RF for robustness and interpretability (importance); XGBoost when you need maximum predictive power and can validate well.

Implementing Return Prediction (Trees)

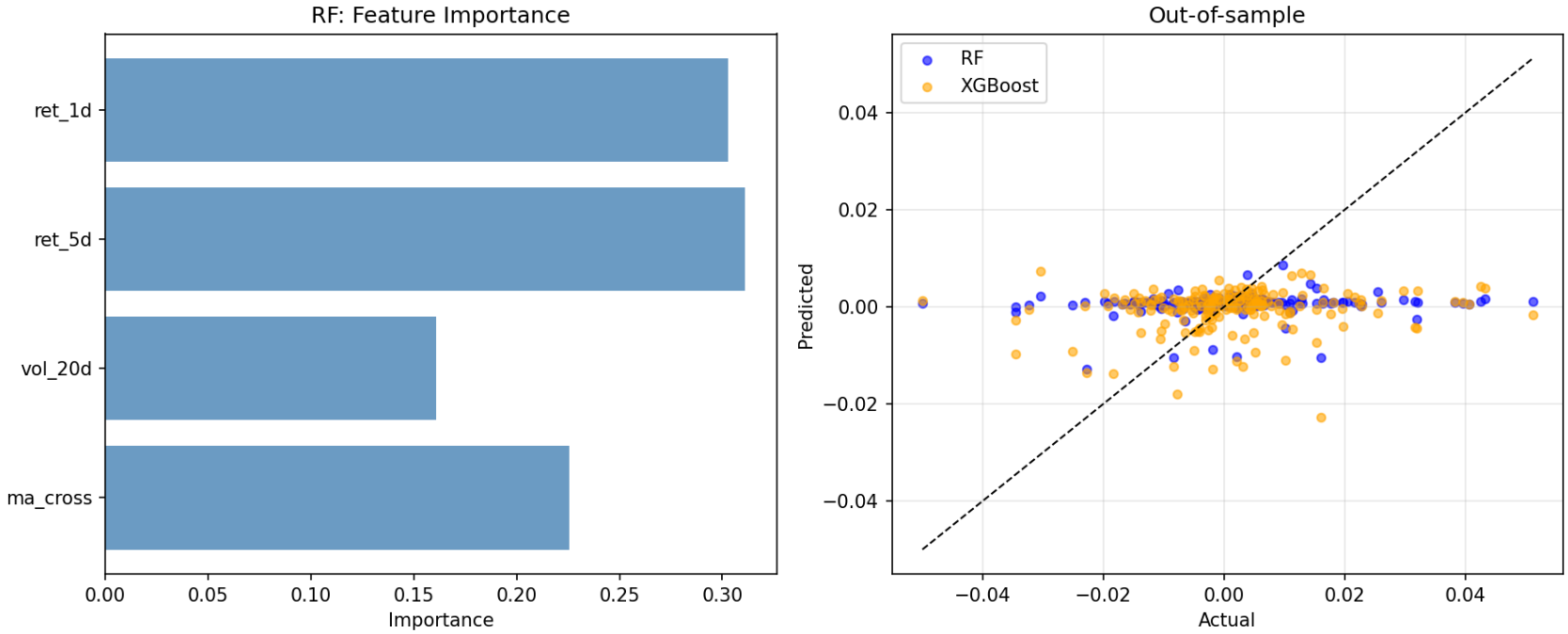
Your AI Prompt

"Write Python code to fit Random Forest and XGBoost models for return prediction. Use features (e.g. past returns, volatility, MA crossover) and a temporal train/test split. Report out-of-sample performance and (for Random Forest) feature importances."

Key Libraries:

- `from sklearn.ensemble import RandomForestRegressor`
- `import xgboost as xgb`

Result: Return Prediction (Trees)



Left: RF feature importance (e.g. ret_1d, vol_20d). **Right:** Out-of-sample predicted vs actual return; RF and XGBoost often similar; points near diagonal = better fit.

Clustering: k-means

Task: Unsupervised grouping (no labels). E.g. group stocks by features, or identify regimes.

k-means: Partition n observations into k clusters by minimizing within-cluster variance.

Strengths: Simple, fast, scalable; works well when clusters are roughly spherical and similar size.

Weaknesses: Must choose k; sensitive to scale (standardize features!); assumes convex clusters; sensitive to initialization.

When to use: Asset/portfolio clustering, regime detection, exploratory analysis. Combine with PCA if many features.

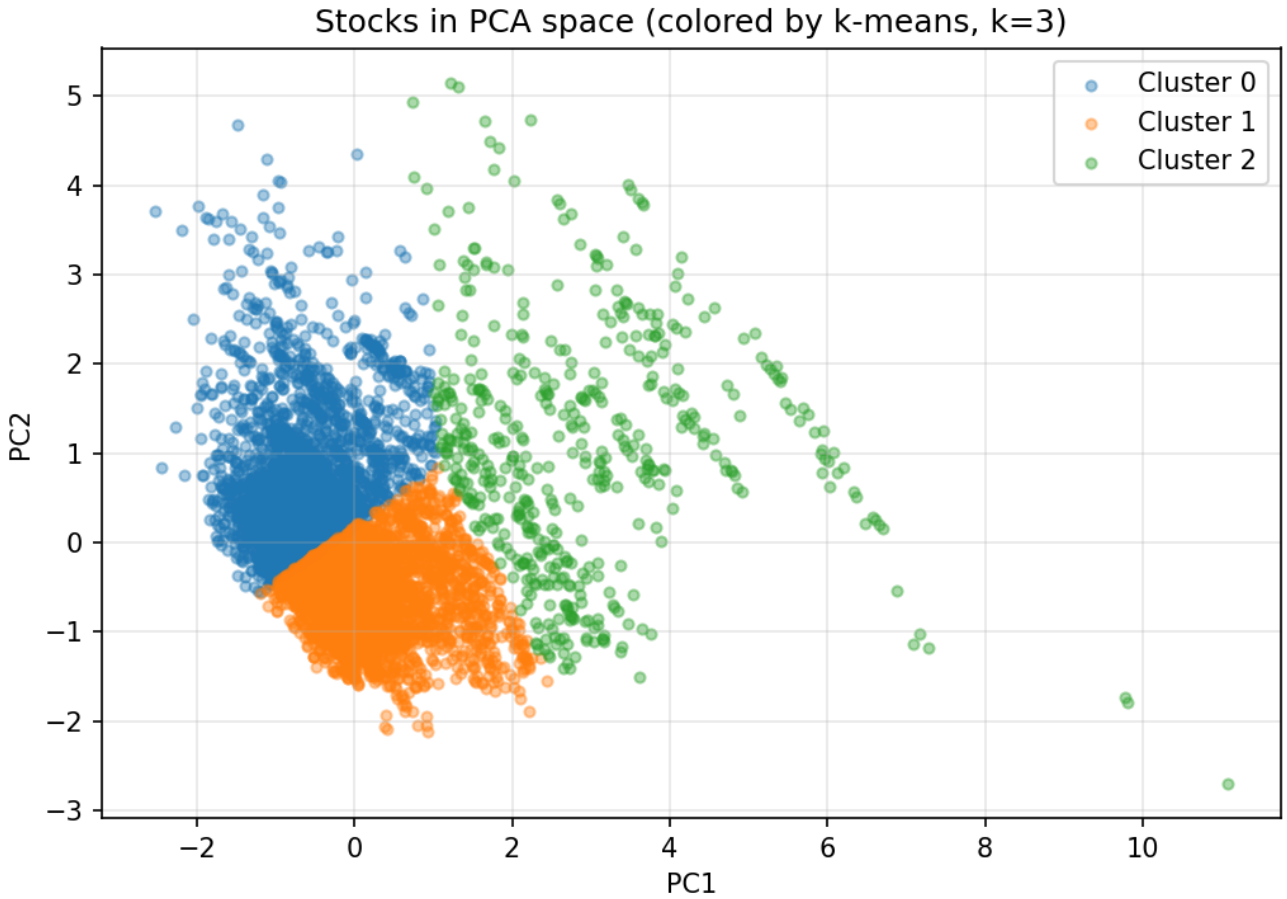
Implementing Clustering

Your AI Prompt

"Write Python code to cluster stocks using k-means. Download daily returns for 20 stocks (e.g. from akshare), compute rolling volatility and momentum for each, standardize features, then fit `KMeans(n_clusters=3)`. Plot the first two PCA components with points colored by cluster label. Discuss whether clusters correspond to sectors or risk profiles."

Key: `sklearn.cluster.KMeans`, `StandardScaler()`, choose k via elbow plot or domain knowledge.

Result: Clustering (k-means)



Points = (stock, date) in first two PCA components of rolling volatility and momentum; colors = cluster label ($k = 3$). Clusters can reflect risk/regime or sector-like groups.

Classification: LDA, Logistic Regression, SVM

Task: Predict a **label** (e.g. return direction up/down, default yes/no).

LDA (Linear Discriminant Analysis): Finds linear combination of features that best separates classes. Interpretable, assumes Gaussian classes with shared covariance.

Logistic Regression: Models probability of class via linear combination + sigmoid. Interpretable (coefficients), no distributional assumption; baseline for binary classification.

SVM (Support Vector Machine): Finds margin-maximizing hyperplane (linear or kernel). Good with few samples, kernel = non-linear; less interpretable, sensitive to tuning.

When to use: LDA/logistic when interpretability and simplicity matter; SVM when you need non-linearity and have moderate data; trees/XGBoost when you care most about raw accuracy.

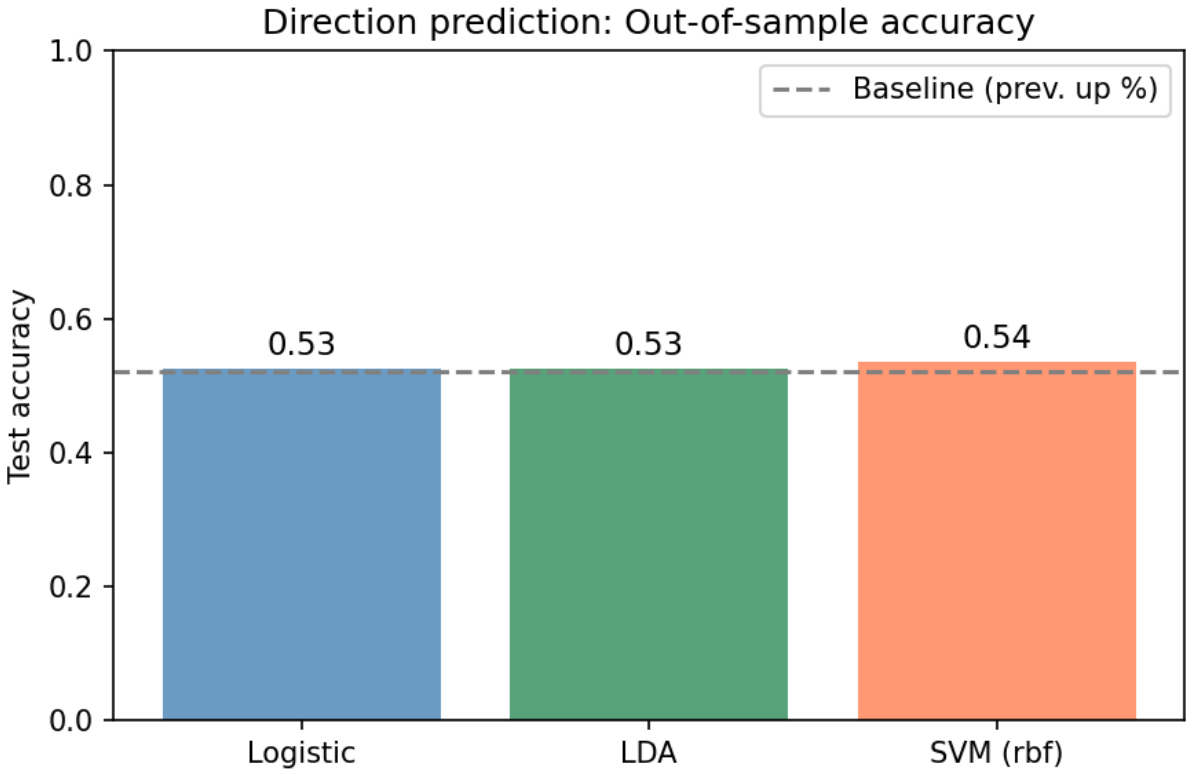
Implementing Classification

Your AI Prompt

"Write Python code to predict next-day return **direction** (up/down) using features (e.g. past returns, volatility, MA crossover). Use `LogisticRegression`, `LinearDiscriminantAnalysis`, and `SVC(kernel='rbf')` from sklearn. Compare accuracy and (if possible) feature coefficients for the linear models. Use a temporal train/test split."

Key: Create binary target (`y = (returns > 0).astype(int)`), avoid look-ahead; use `LogisticRegression`, `LinearDiscriminantAnalysis`, `sklearn.svm.SVC`.

Result: Direction Classification



Example out-of-sample accuracy: Logistic 52.6%, LDA 52.6%, SVM (rbf) 53.6%. Direction is hard to predict; accuracy near 50% is common. Compare to a baseline (e.g. fraction of up days).

When to Use Which Method

Task	Method	Strengths	Use when
Regression (return level)	ARMA	Interpretable, stable	Need coefficients, baseline
Regression	Random Forest	Non-linear, robust	Default choice for tabular
Regression	XGBoost	Often best accuracy	Prioritize accuracy
Classification (direction)	Logistic, LDA	Interpretable	Need coefficients, baseline
Classification	SVM	Non-linear (kernel)	Margin focus
Classification	Trees/XGBoost	Flexible, accurate	Prioritize accuracy
Clustering (no labels)	k-means	Simple, fast	Regimes, asset grouping

Takeaway: Start simple (linear/logistic); add trees or SVM if needed. Use clustering for exploration or regime/group structure. Always validate out-of-sample and avoid look-ahead.

Key Takeaways

What We Learned:

1. Feature engineering is crucial
2. Avoid look-ahead bias
3. Proper train-test splits for time series
4. Multiple ML models available
5. Interpretation is important
6. Most attempts fail - that's normal

Realistic Expectations

What to Expect:

- Many features won't help
- Complex models often overfit
- Out-of-sample worse than in-sample

Success Looks Like:

- Modest but consistent out-of-sample improvement
- Beats simple benchmarks
- Survives walk-forward testing
- Profitable after transaction costs

Be Humble

If it was easy, everyone would be rich!

Next Lecture Preview

Slide 6: Alternative Data & Modern Applications

Beyond traditional data:

- News sentiment
- Alternative data sources
- Event studies
- Dashboards

Exploring new frontiers!